

The Agent Identity Manager (AIM): Accumulating Trust for AI Agent Authentication

Authors Anonymized

Abstract—The rapid spread of autonomous AI agents based on large language models (LLMs) has highlighted the lack of an infrastructure that can reliably authenticate these agents. Existing authentication mechanisms (API keys, OAuth, JWT) depend on central servers, carry single point of failure risk and do not provide a standard mechanism for agent-to-agent authentication. Cryptographic accumulators, on the other hand, are a mature cryptographic primitive for set membership verification but have not yet been applied to AI agent authentication. In this work, we propose the Agent Identity Manager (AIM) framework, which uses a multiple cuckoo filter-based cryptographic accumulator architecture. AIM is supported by a dedicated application layer protocol (AIM Protocol) that standardizes agent registration, verification, revocation and synchronization operations. Accumulating the same agent identities into multiple filters with different hash functions reduces the combined false positive rate. The inherent false positive risk of this data structure is managed through a two-stage verification strategy (fast local check and central exact verification for critical operations). The proposed work presents the first system architecture that applies symmetric accumulator data structures to AI agent authentication and provides a foundational starting point for future research in this area.

Keywords— *AI agent authentication, cryptographic accumulator, cuckoo filter, multi-agent systems*

I. INTRODUCTION

Autonomous AI agents based on large language models (LLMs) are spreading rapidly across a broad range of domains, from software development to customer service and from financial analysis to scientific research. Frameworks such as LangChain [1], AutoGen [2] and CrewAI [3] facilitate the development of multi-agent systems in which multiple agents work in coordination and significantly accelerate the industrial adoption of such systems. However, this rapid growth has also brought with it a critical infrastructure gap: no mechanism exists to reliably verify whether an agent actually has the identity it claims, whether it is authorized and whether its access can be instantly revoked when needed.

Industry reports reveal the scale of the identity and access gap with concrete data. According to Gartner's projections, by the end of 2026, 40% of enterprise applications will include task-specific AI agents, up from less than 5% in 2025 [4]. At the same time, a survey conducted by the Cloud Security Alliance in January 2026 with 228 IT and security professionals showed that 68% of organizations are unable to distinguish AI agent activity from human activity and that 74% grant their agents more access privileges than necessary [5]. According to another survey carried out by the same organization between September and October 2025 with 285 participants, only 21% of organizations are able to maintain a real-time inventory of their agents and 84% believe they would fail a compliance audit focused on agent behavior [6]. These data demonstrate that agent identity management is no longer a theoretical discussion but has become an urgent operational requirement.

At the root of this problem lie the architectural shortcomings of existing authentication mechanisms. Common approaches such as static API keys, OAuth tokens and JWT (JSON Web Token) are designed for human users and traditional software services. These mechanisms depend on a central server for every authentication operation; this both creates a single point of failure risk and leads to scalability issues as the agent population grows. More importantly, there is no standard mechanism for agent-to-agent authentication, in which two agents can verify each other without consulting a central authority.

In addressing these shortcomings, cryptographic accumulators, which enable verification of set membership through a compact proof, emerge as a promising alternative. The first cryptographic accumulator was proposed by Benaloh and De Mare [7] in 1993. This data structure has been successfully applied in areas such as certificate revocation [8] and anonymous authentication [9] in various forms including RSA accumulators, Merkle trees and bilinear map-based constructions. However, our extensive literature review has revealed that cryptographic accumulators have not been evaluated in the context of AI agent authentication.

Protocols such as LDAP [10], OpenID Connect [11] and RADIUS [12], which are widely used in current authentication systems, operate based on centralized identity providers and create latency and bandwidth issues as the number of users grows. To overcome this problem, Çekiş et al. [13] proposed an edge authentication protocol that shifts the verification process from the central server to edge devices and provides $O(1)$ -complexity authentication based on a cryptographic accumulator. On another front, working to extend the accuracy boundaries of probabilistic data structures, Medury et al. [14] designed a structure that applies a cascading algorithm to cuckoo filters and provides zero-false-positive membership testing in cases where all potential false positives are known. The proposed AIM framework builds on the foundations laid by these two studies: it extends the vision of accumulator-based distributed authentication [13] into the AI agent ecosystem, while adapting the multiple cuckoo filter architecture [14] to reduce the false positive rate multiplicatively.

In this study, we propose Agent Identity Manager (AIM), a new framework for AI agent authentication that uses a multiple cuckoo filter-based cryptographic accumulator architecture. AIM offers a balanced architecture between a centralized identity authority and agent authentication at the edge. At the core of the system lies the idea of accumulating the same agent credentials in multiple cuckoo filters [15] using different hash functions. This structure reduces the combined false positive rate of k independent filters to p^k , where p is the false positive rate of a single filter. The false positive risk inherent to symmetric accumulators is minimized within the AIM framework through the parallel operation of multiple symmetric accumulators. In addition, when needed in critical operations, a second verification can be performed against the authoritative record database (Agent Registry) on the AIM server, thereby preventing a possible false positive from

turning into actual unauthorized access. The distribution of filter copies to each agent enables local verification with minimum network latency and minimizes dependence on a central authority in agent-to-agent scenarios.

The AIM framework standardizes agent registration, verification, revocation and filter synchronization operations through a dedicated application layer protocol (AIM Protocol). The main contribution of this work is the design of the first system architecture that applies a multiple cuckoo filter-based cryptographic accumulator structure to AI agent authentication, together with the definition of the AIM Protocol framework that supports this architecture.

The rest of the paper is organized as follows. Section II reviews the relevant studies. Section III presents preliminaries on cuckoo filters and cryptographic accumulators. Section IV provides the technical details of the AIM framework and the AIM Protocol. Section V examines the outcomes and outlines future research directions.

II. RELATED WORKS

This section presents the state of the art in three main research areas related to the proposed work: AI agent security and identity management, authentication applications of cryptographic accumulators and the use of cuckoo filters in the security domain. After evaluating the current state of each area, the literature gap at the intersection of these fields is presented.

AI agent identity management has become a rapidly developing area at both academic and industrial levels since 2024. Chan et al. [16] discussed agent identifiers, real-time monitoring and activity logging mechanisms aimed at improving the visibility of AI agents; the same group's "IDs for AI Systems" work [17] proposed a conceptual framework for assigning identities to AI system instances. In terms of cryptographic approaches, Rodriguez Garzon et al. [18] presented the first working prototype that provides self-sovereign identity by assigning W3C DIDs and Verifiable Credentials to AI agents; they demonstrated technical feasibility through LangChain and AutoGen integration but also revealed the limitations of an LLM managing security procedures on its own. AgentCrypt [19] proposed a four-layer privacy framework based on homomorphic encryption for agent-to-agent communication. On the industry side, the OWASP Agentic AI Top 10 (December 2025) established the first systematic risk taxonomy for autonomous agents and identified identity abuse as one of the main risk categories [20]; NIST NCCoE is running an active project titled "Software and AI Agent Identity and Authorization" [21]. Microsoft's Agent Governance Toolkit (April 2026) includes DID-based Ed25519 cryptographic identity and the Inter-Agent Trust Protocol [22], but it relies on traditional PKI primitives.

While the majority of these approaches rely on traditional cryptographic infrastructures, cryptographic accumulators, which enable verification of set membership through a compact proof, offer a lighter and more distributable alternative in the authentication domain. Since their introduction by Benaloh and De Mare [7] in 1993 as one-way accumulators, these structures have been applied in various authentication scenarios. Camenisch and Lysyanskaya [9] defined dynamic accumulators, enabling element addition/deletion at a cost independent of set size and membership verification through zero-knowledge proofs. Ren

et al. [23] systematically examined the evolution of accumulator schemes; Özçelik et al. [24] presented a symmetric and asymmetric classification of accumulators and comparatively analyzed the space, time complexity and communication cost of each structure. Loporchio et al. [25] evaluated accumulator applications in blockchain systems. In terms of authentication applications, Özçelik and Skjellum [8] used RSA accumulators for blockchain-based distributed certificate revocation in their CryptoRevocate work; DA³4FL [26] combined dynamic accumulators with blockchain for authentication purposes in federated learning environments. More recently, Çekiş et al. [13] proposed a cryptographic accumulator-based edge authentication protocol. This study uses an ECC-based dynamic asymmetric accumulator to shift the verification process from the central server to edge devices and provides $O(1)$ -complexity authentication. In the protocol design, finite state machines and sequence diagrams are used to formally model the system. However, this work does not target the AI agent ecosystem and carries the witness management cost arising from the use of an asymmetric accumulator. Our proposed AIM framework shares a similar distributed authentication vision but offers a different approach: through its choice of a symmetric accumulator (cuckoo filter), it requires no witnesses, supports deletion and reduces the false positive rate multiplicatively through a multiple filter architecture.

Cuckoo filters, which stand out within the symmetric accumulator class, have been preferred in authentication particularly in environments with scalability and low-latency requirements. Fan et al. [15] proposed this structure in 2014 as a practical alternative to Bloom filters. The authentication applications of cuckoo filters have predominantly concentrated on vehicular networks: Cui et al. [27] designed a privacy-preserving authentication protocol with their SPACF scheme; Li et al. [28] demonstrated the deletion advantage of cuckoo filters in pseudonym certificate revocation; Xenakis et al. [29] extended the 5G-AKA protocol with cuckoo filters to improve batch authentication efficiency. In the healthcare domain, Moni and Gupta [30] performed mutual anonymous authentication with a dual positive-negative cuckoo filter architecture in remote patient monitoring networks and provided formal verification with AVISPA. While all of these works demonstrate the effectiveness of cuckoo filters in authentication, they all target traditional network environments such as VANET, 5G and IoMT.

In our extensive review, no academic work has been found that applies cryptographic accumulator structures or probabilistic data structures in the context of AI agent authentication. Existing AI agent identity solutions (DID+VC [18], zkVM, OAuth extensions) provide exact cryptographic verification but are heavy in terms of computational and communication cost and are not suitable for microsecond-level local verification in agent-to-agent scenarios. On the other hand, cuckoo filters have been successfully used in VANET and IoMT environments but AI agent scenarios have not been evaluated. This work aims to fill this gap: it presents an AI agent authentication framework based on a multiple cuckoo filter architecture that performs $O(1)$ -complexity local verification and provides instant revocation through deletion support.

III. BACKGROUND

This section outlines the foundation of the proposed system, detailing the general concept of cryptographic

accumulators alongside their specific implementation as cuckoo filters.

A. Cryptographic Accumulators

A cryptographic accumulator is a space- and time-efficient data structure that represents the elements of a set with a single fixed-size value and enables membership in this set to be verified efficiently [24]. The core idea is to accumulate the elements of a set $A = \{y_1, y_2, \dots, y_n\}$ into a small value z , where only set members can be proven to have been accumulated [31].

Cryptographic accumulators can be divided into two main classes: symmetric and asymmetric [24]. Symmetric accumulators are built on symmetric cryptographic primitives and can verify set membership directly without requiring a witness [32]. Bloom filters [33] and cuckoo filters [15] fall into this class; they store element fingerprints in an array structure using k hash functions and allow $O(1)$ -complexity queries. However, they can produce false positives, meaning they may incorrectly report an element that is not in the set as a member. Asymmetric accumulators, on the other hand, are built on asymmetric cryptographic primitives and quasi-commutative hash functions [7]. RSA accumulators [7], bilinear map-based constructions [34] and Merkle trees [35] belong to this class. In these structures, a witness value is generated for each element and verification is performed as $\text{Ver}(k, z, y_i, w_i) \rightarrow \{\text{TRUE}, \text{FALSE}\}$ [31]. They do not produce false positives but witness generation and updating carry computational cost.

Accumulators are also classified by the operations they support: dynamic accumulators efficiently support element addition and deletion [9], while static structures do not support these operations. By the type of membership proof, positive accumulators support only membership, negative accumulators support only non-membership and universal accumulators support both proof types [24], [36]. In this study, cuckoo filters from the dynamic and positive symmetric accumulator class are preferred.

B. Cuckoo Filters

The cuckoo filter is a probabilistic data structure proposed by Fan et al. [15] in 2014 that performs approximate membership queries. The structure consists of a bucket array based on the cuckoo hashing principle [15]. Each element is represented by an f -bit fingerprint placed in one of two possible bucket positions. When an element is being inserted, if the primary bucket is full, the existing element is moved to its secondary bucket. This is the displacement mechanism of cuckoo hashing, which optimizes space efficiency [24]. During a query, the two possible buckets of the element are checked for the presence of its fingerprint. If there is no match, the element is definitely not in the set; if there is a match, the element is likely to be in the set.

The false positive rate of a single cuckoo filter is determined by the fingerprint length f and the number of entries per bucket b . For an f -bit fingerprint and a bucket structure with $b = 4$ entries, this rate is approximately expressed as [15]:

$$\epsilon \approx 2b/2^f \quad (1)$$

Fan et al. [15] experimentally showed that, for a given number of elements, the cuckoo filter offers a lower false

positive rate and space cost compared to the space-optimized Bloom filter. Unlike Bloom filters, cuckoo filters are not static accumulators; they support element deletion [24]. Bloom filters, on the other hand, are static accumulators: because multiple elements share the same bit positions, element deletion is not directly possible and requires the entire filter to be rebuilt [24], [33].

In the multiple filter approach proposed in this study, the same set of elements is added to k independent cuckoo filters using different hash functions. Since a match in all k filters is required for an element to be considered valid, the combined false positive rate is:

$$\epsilon_{\text{combined}} \approx \epsilon^k \quad (2)$$

This multiplicative reduction enables the combined rate to drop to 10^{-6} when $k = 3$ filters are used with a single-filter rate of $\epsilon = 0.01$. The primary rationale for selecting cuckoo filters in the context of AI agent authentication lies in their combined advantages. Specifically, their deletion support eliminates the need to rebuild the filter when revoking an agent's authorization, their compact size allows copies to be distributed to the agent side for local verification, and they offer an $O(1)$ -complexity query time.

IV. SYSTEM DESIGN

A. System Overview

The Agent Identity Manager (AIM) framework proposed in this study is designed to perform AI agent authentication using cryptographic accumulator architecture.

B. Working Principle

At the core of the AIM framework lies a multiple cuckoo filter-based accumulator structure. The working principle of this structure is shown in Fig. 1.

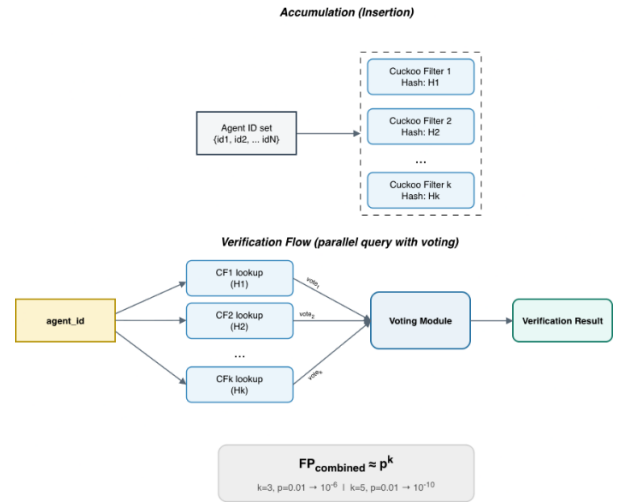


Fig. 1. Working principle of the multiple cuckoo filter-based accumulator.

In the system, each registered agent ID is added to k independent cuckoo filters with different hash functions ($H_1, H_2, \dots, H_k$). During the verification process, the queried agent_id is evaluated through the multiple filter structure, and the combined output of the filters forms the basis of the verification decision.

The main advantage of this multiple filter strategy is the multiplicative reduction of the false positive rate. If the false

positive rate of a single cuckoo filter is p and k independent filters are used, the probability of a simultaneous false positive across all filters becomes p^k . A key structural advantage of the cuckoo filter over the Bloom filter [33] is that it natively supports element deletion [15], [24]. This feature is critically important in agent revocation scenarios: the ID of a revoked agent can be deleted from all filters in $O(1)$ time without the need to rebuild the filter.

C. System Components

The AIM framework consists of four main components. These components and the interactions between them are shown in Fig. 2.

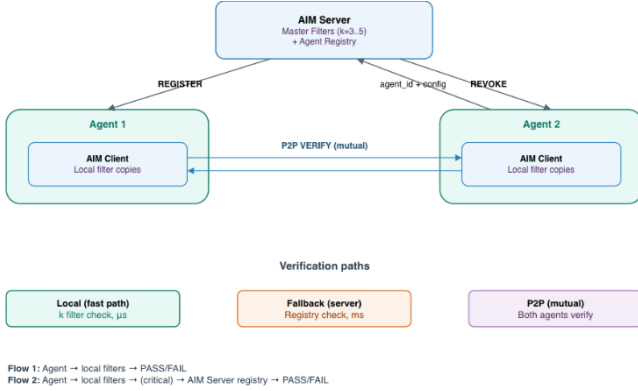


Fig. 2. AIM framework components and their connections.

AIM Server: This is the identity authority. It is responsible for agent ID generation, the management of master cuckoo filters (k filters) and the maintenance of the Agent Registry. All agent registration, revocation and filter synchronization operations are coordinated through this component.

Master Filters: These are k redundant cuckoo filters located on the AIM Server. Each one accumulates the same agent ID set using different hash functions. These filters are updated during registration and revocation operations, and their updated copies are distributed to all AIM Clients.

Agent Registry: This is the complete and exact database of all agent IDs. Since cuckoo filters are probabilistic structures, the registry is used as a fallback source in situations where exact verification is required (critical operations).

AIM Client: This is the client module embedded in each agent's runtime environment. It maintains a local copy of the master filters and performs verification operations. Verification is performed through the accumulators within the AIM Client, enabling fast verification without the need for network communication. In critical operations or in situations where local verification may be insufficient, the AIM Client can contact the central AIM Server to perform exact verification through the Agent Registry; this fallback mechanism allows the risk inherent to the probabilistic structure to be managed.

The system supports two main verification paths. (1) Local verification: fast verification performed through the AIM Client's local filter copies, which does not require network communication. P2P mutual verification is also possible through this path: when two agents wish to communicate with each other, both sides can verify the identity of the other party using their own local filter copies without consulting a central

authority. (2) Central fallback: exact verification performed through the Agent Registry on the AIM Server during critical operations.

D. System Workflow

The system workflow can be examined in two main phases: registration and verification. This flow is shown as a sequence diagram in Fig. 3.

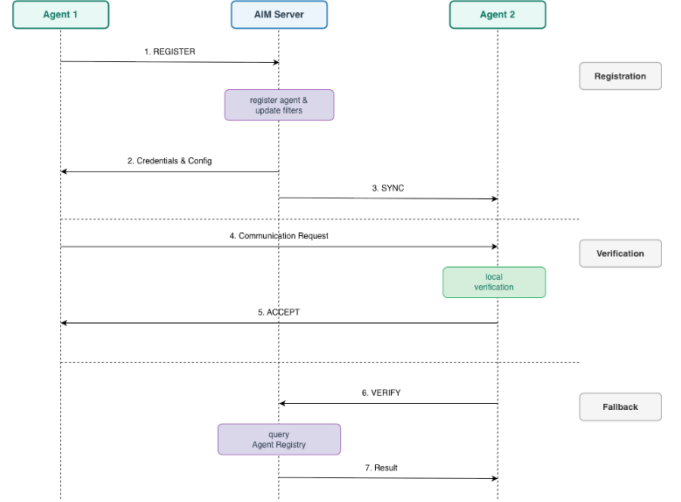


Fig. 3. AIM framework workflow.

1) Registration Flow

When a new agent is to be registered with the system, the following steps are followed:

- The agent sends a REGISTER request to the AIM Server together with its metadata.
- The AIM Server validates the request and generates a unique identity for the agent, recording it in the Agent Registry and adding it to all k master cuckoo filters using different hash functions.
- The AIM Server returns the necessary credentials and configuration parameters to the agent so that it can later participate in verification operations.
- The updated filters are propagated to the connected AIM Clients through the synchronization mechanism, ensuring that local filter copies remain consistent across the system.

2) Verification Flow

Two different flows exist for verifying an agent's identity:

Local Verification (Flow 1): This is the fastest and most frequently used path. The AIM Client of the verifying agent checks the queried agent_id in all k local cuckoo filters. Based on the combined output of the filters, the agent is evaluated as authorized or unauthorized; the use of multiple filters reduces the false positive probability to p^k . Since this flow operates through local filter copies, it provides the verification path with the lowest latency and does not require any network communication. The same flow also covers the P2P mutual verification scenario: when two agents wish to communicate with each other, both sides verify the other party's agent_id using their own local filter copies. This symmetric verification takes place without dependence on a central authority and

benefits from the distributability advantage provided by the compact structure of cuckoo filters.

Central Fallback (Flow 2): In critical operations (sensitive data access, highly privileged resources) or in situations where local verification may be insufficient, the AIM Client sends a VERIFY message to the central AIM Server. The AIM Server performs an exact check against the Agent Registry and completes the verification with a challenge-response. This fallback mechanism makes it possible to manage the risk inherent to the probabilistic structure and to ensure reliability in scenarios that require exact verification.

E. AIM Protocol

The AIM Protocol is an application layer protocol that standardizes all communication between the AIM Server and AIM Clients. It operates over TCP sockets and defines four main message types: REGISTER, VERIFY, REVOKE and SYNC. The protocol supports both persistent (long-lived connection, multiple messages) and non-persistent (one-time connection) modes.

TLS is designed as an optional security extension that can be added to the transport layer. Filter synchronization is achieved through two mechanisms: periodic pull (client-initiated at predefined intervals) and urgent push (server-initiated after a REVOKE operation). This hybrid model minimizes the time a revoked agent remains as a ghost.

V. DISCUSSION AND CONCLUSION

The rapid spread of LLM-based autonomous AI agents has highlighted the lack of an infrastructure that can reliably authenticate agents. Industry surveys reveal that the majority of organizations cannot distinguish agent activity from human activity and would fail compliance audits focused on agent behavior [5], [6]. Existing mechanisms such as static API keys, OAuth and JWT suffer from architectural shortcomings such as dependence on a central server, single point of failure risk and the lack of a standard mechanism for agent-to-agent authentication. Cryptographic accumulators, on the other hand, offer a mature primitive that enables verification of set membership through a compact proof, yet these structures have not been applied to AI agent authentication.

To fill this gap, we propose Agent Identity Manager (AIM), a new framework for AI agent authentication that uses a multiple cuckoo filter-based cryptographic accumulator architecture. The main contribution of AIM is the definition of a system architecture that applies symmetric accumulator data structures to AI agent authentication, together with the AIM Protocol framework supporting this architecture. In this respect, AIM serves as a foundation that can be built upon; by establishing a bridge between the fields of cryptographic accumulators and AI agent security, it provides a starting point for both academic and practical extensions. Three main principles stand out in the design of the system: (i) multiplicative reduction of the false positive rate to p^k through the multiple cuckoo filter architecture; (ii) management of the risk inherent to the symmetric accumulator structure through a two-stage verification strategy (fast preliminary check at local filters and exact verification through the Agent Registry for critical operations); (iii) execution of agent-to-agent authentication without dependence on a central authority through the distribution of filter copies to each agent. This design extends the accumulator-based distributed

authentication vision proposed by Çekiş et al. [13] to the AI agent ecosystem, while leveraging the false positive reduction potential of the multiple cuckoo filter approach demonstrated by Medury et al. [14].

When the expected outcomes of the proposed architecture are evaluated, AIM is expected to provide low verification time and zero network latency through local verification compared to current centralized approaches. At the same time, this work also has some open points. The view that the false positive value is reduced through the use of parallel symmetric accumulators rests on the assumption of independence between the filters, and this assumption needs to be experimentally verified. A revoked agent temporarily remaining in a ghost state during the window between push-based synchronization and the update of local filters is a limitation inherent to the eventual consistency model. In addition, the presence of local filter copies on the agent side raises new threat models such as enumeration attacks.

As future work, we aim to develop a Python-based prototype of the proposed architecture and conduct a comparative experimental evaluation with current methods. Performance measurements in large-scale scenarios and synchronization window analysis in revocation scenarios are also planned. Other research directions include multi-dimensional filter verification where each filter checks a different identity dimension (organization, authority level), the evolution of the central AIM server into a federated or fully decentralized structure, proving group membership without revealing agent identity through zero-knowledge proof integration and binding agent IDs to hardware security modules such as TPM or secure enclaves. In this regard, AIM provides a foundational starting point for the maturation of authentication infrastructure in multi-agent ecosystems and, through extensions to be built upon the proposed architecture, opens the way for the development of more extensive solutions in this area.

REFERENCES

- [1] LangChain, “LangChain: Build context-aware reasoning applications,” 2024. [Online]. Available: <https://www.langchain.com>
- [2] Q. Wu et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv:2308.08155, 2023.
- [3] CrewAI, “CrewAI: Framework for orchestrating role-playing autonomous AI agents,” 2024. [Online]. Available: <https://www.crewai.com>
- [4] Gartner, Inc., “Gartner predicts 40% of enterprise apps will feature task-specific AI agents by 2026, up from less than 5% in 2025,” Press Release, Aug. 26, 2025.
- [5] Cloud Security Alliance, “The identity and access gaps in the age of autonomous AI,” Survey Report, commissioned by Aembit, Mar. 2026.
- [6] Cloud Security Alliance, “Securing autonomous AI agents,” Survey Report, commissioned by Strata Identity, Feb. 2026.
- [7] J. Benaloh and M. de Mare, “One-way accumulators: A decentralized alternative to digital signatures,” in Proc. EUROCRYPT ’93, Springer LNCS, vol. 765, 1993, pp. 274–285.
- [8] İ. Özçelik and A. Skjellum, “CryptoRevoke: A cryptographic accumulator based distributed certificate revocation list,” in Proc. 11th IEEE CCWC, 2021, pp. 865–872.
- [9] J. Camenisch and A. Lysyanskaya, “Dynamic accumulators and application to efficient revocation of anonymous credentials,” in Proc. CRYPTO 2002, Springer LNCS, vol. 2442, 2002, pp. 61–76.
- [10] V. Koutsonikola and A. Vakali, “LDAP: Framework, practices, and trends,” IEEE Internet Comput., vol. 8, no. 5, pp. 66–72, 2004.
- [11] D. Fett, R. Küsters, and G. Schmitz, “The web SSO standard OpenID Connect: In-depth formal security analysis and security guidelines,” in Proc. IEEE 30th CSF, 2017, pp. 189–202.

- [12] S. Goldberg et al., "RADIUS/UDP considered harmful," in Proc. 33rd USENIX Security, 2024, pp. 7429–7446.
- [13] İ. K. Çekiş, A. Toros, V. Yiğit, and İ. Özçelik, "Edge authentication using a cryptographic accumulator," in Proc. 2025 9th Int. Symp. Multidisciplinary Studies and Innovative Technologies (ISMSIT), IEEE, 2025, pp. 1–5.
- [14] S. Medury, A. Altarawneh, and A. Skjellum, "Design and evaluation of cascading cuckoo filters for zero-false-positive membership services," in Proc. 11th IEEE CCWC, 2021, pp. 1061–1065.
- [15] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than Bloom," in Proc. 10th ACM CoNEXT, 2014, pp. 75–88.
- [16] A. Chan et al., "Visibility into AI agents," in Proc. ACM FAccT '24, 2024, pp. 958–973.
- [17] A. Chan et al., "IDs for AI systems," arXiv:2406.12137, 2024.
- [18] S. Rodriguez Garzon et al., "AI agents with decentralized identifiers and verifiable credentials," arXiv:2511.02841, 2025.
- [19] H. Karthikeyan et al., "AgentCrypt: Advancing privacy and secure computation in AI agent collaboration," in Proc. NeurIPS, 2025.
- [20] OWASP, "Top 10 for Agentic Applications 2026," Dec. 2025. [Online]. Available: <https://genai.owasp.org>
- [21] R. Galluzzo, B. Fisher, H. Booth, and J. Roberts, "Accelerating the adoption of software and AI agent identity and authorization," NIST NCCoE Concept Paper, Feb. 2026.
- [22] Microsoft, "Agent Governance Toolkit," GitHub, MIT License, Apr. 2026. [Online]. Available: <https://github.com/microsoft/agent-governance-toolkit>
- [23] Y. Ren, X. Liu, Q. Wu, L. Wang, and W. Zhang, "Cryptographic accumulator and its application: A survey," Security and Communication Networks, vol. 2022, Article 5429195, 2022.
- [24] İ. Özçelik, S. Medury, J. Broadus, and A. Skjellum, "An overview of cryptographic accumulators," in Proc. 18th Int. Conf. Security and Cryptography (SECRYPT), 2021, pp. 661–668.
- [25] M. Loporchio, A. Bernasconi, D. Di Francesco Maesa, and L. Ricci, "A survey of set accumulators for blockchain systems," Computer Science Review, vol. 49, Article 100570, 2023.
- [26] DA³FL authors, "DA³FL: A robust dynamic accumulator-based authentication and key agreement with preserving model training data integrity for federated learning," Nature Scientific Reports, Article s41598-025-33685-1, 2025.
- [27] J. Cui, J. Zhang, H. Zhong, and Y. Xu, "SPACF: A secure privacy-preserving authentication scheme for VANET with cuckoo filter," IEEE Trans. Veh. Technol., vol. 66, no. 11, pp. 10283–10295, 2017.
- [28] Li et al., "Improving efficiency of pseudonym revocation in VANET using cuckoo filter," in Proc. IEEE Conf., 2020.
- [29] C. Xenakis et al., "Lightweight and space-efficient vehicle authentication based on cuckoo filter," in Proc. IEEE Conf., 2020.
- [30] S. S. Moni and D. Gupta, "Secure and efficient privacy-preserving authentication scheme using cuckoo filter in remote patient monitoring network," in Proc. IEEE TPS-ISA, 2022, pp. 208–216.
- [31] N. Fazio and A. Nicolosi, "Cryptographic accumulators: Definitions, constructions and applications," Technical Report, New York University, 2002.
- [32] A. Kumar, P. Lafourcade, and C. Lauradoux, "Performances of cryptographic accumulators," in Proc. 39th Annual IEEE Conf. Local Computer Networks, 2014, pp. 366–369.
- [33] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," Communications of the ACM, vol. 13, no. 7, pp. 422–426, 1970.
- [34] C. Tartary, S. Zhou, D. Lin, H. Wang, and J. Pieprzyk, "Analysis of bilinear pairing-based accumulator for identity escrowing," IET Information Security, vol. 2, no. 4, pp. 99–107, 2008.
- [35] R. C. Merkle, "A certified digital signature," in Proc. Conf. Theory and Application of Cryptology, 1989, pp. 218–238.
- [36] J. Li, N. Li, and R. Xue, "Universal accumulators with efficient nonmembership proofs," in Proc. ACNS, 2007, pp. 253–269.
- [37] SailPoint, "AI agents: The new attack surface. A global survey of security, IT professionals and executives," Research Report, May 2025.